

Lab 6 Report

Introduction to OpenMP

Name: Huzaifa Arshad

Roll No: 2023-CS-86

Course: Parallel and Distributed Computing

University of Engineering and Technology (UET)
Lahore

Contents

1	Objectives	2
2	Theory	3
2.1	Key Concepts	3
3	Exercise 1: Hello World with OpenMP	4
3.1	Description	4
3.2	Commands Used	4
3.3	Output	4
3.4	Observation	4
4	Exercise 2: Parallel Counting	5
4.1	Description	5
4.2	Commands Used	5
4.3	Results	5
4.4	Output	6
4.5	Observation	6
5	Exercise 3: Critical Section	7
5.1	Description	7
5.2	Commands Used	7
5.3	Output	7
5.4	Observation	7
6	Analysis	8
7	Conclusion	9

Chapter 1

Objectives

- Understand the OpenMP programming model.
- Learn to compile and execute OpenMP programs.
- Implement parallel programming using OpenMP directives.
- Compare performance of sequential and parallel execution.

Chapter 2

Theory

OpenMP is an API for shared memory parallel programming. It follows the fork-join model where a master thread creates multiple threads to execute tasks in parallel and then joins back.

2.1 Key Concepts

- Parallel Regions
- Work Sharing
- Reduction Clause
- Critical Section

Chapter 3

Exercise 1: Hello World with OpenMP

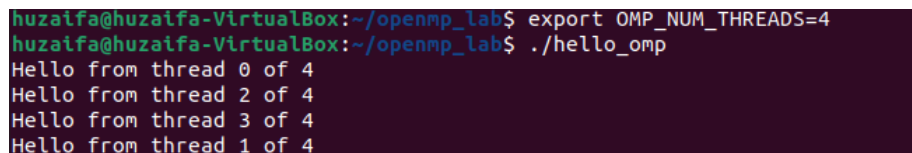
3.1 Description

A simple program where each thread prints its ID and total number of threads.

3.2 Commands Used

```
gcc -fopenmp -o hello_omp hello_omp.c
export OMP_NUM_THREADS=4
./hello_omp
```

3.3 Output

A terminal window with a dark purple background. The prompt is 'huzaiifa@huzaiifa-VirtualBox:~/openmp_lab\$'. The first command is 'export OMP_NUM_THREADS=4'. The second command is './hello_omp'. The output consists of four lines: 'Hello from thread 0 of 4', 'Hello from thread 2 of 4', 'Hello from thread 3 of 4', and 'Hello from thread 1 of 4'.

```
huzaiifa@huzaiifa-VirtualBox:~/openmp_lab$ export OMP_NUM_THREADS=4
huzaiifa@huzaiifa-VirtualBox:~/openmp_lab$ ./hello_omp
Hello from thread 0 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
Hello from thread 1 of 4
```

Figure 3.1: Output of Exercise 1

3.4 Observation

Threads execute in parallel and output order may vary.

Chapter 4

Exercise 2: Parallel Counting

4.1 Description

Counting occurrences of a value in an array using OpenMP reduction.

4.2 Commands Used

```
gcc -fopenmp -O2 -o count_omp count_omp.c
export OMP_NUM_THREADS=4
./count_omp
```

4.3 Results

Threads	Execution Time (seconds)
1	0.07
2	0.04
4	0.03

4.4 Output

```
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ export OMP_NUM_THREADS=4
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ ./count_omp
Count = 999652
Time = 0.20 seconds
Threads used = 4
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ export OMP_NUM_THREADS=1
./count_omp
Count = 1000974
Time = 0.07 seconds
Threads used = 1
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ export OMP_NUM_THREADS=2
./count_omp
Count = 998275
Time = 0.04 seconds
Threads used = 2
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ export OMP_NUM_THREADS=4
./count_omp
Count = 997409
Time = 0.03 seconds
Threads used = 4
```

Figure 4.1: Output of Exercise 2

4.5 Observation

Increasing threads reduces execution time, showing parallel speedup.

Chapter 5

Exercise 3: Critical Section

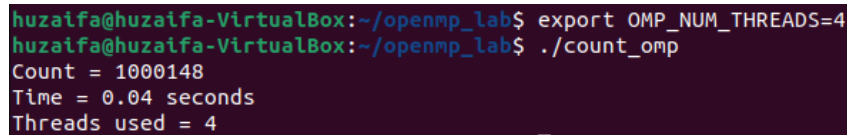
5.1 Description

Using a critical section instead of reduction to update shared variable.

5.2 Commands Used

```
gcc -fopenmp -O2 -o count_omp count_omp.c
export OMP_NUM_THREADS=4
./count_omp
```

5.3 Output

A terminal window with a dark background and light-colored text. The prompt is 'huzaifa@huzaifa-VirtualBox:~/openmp_lab\$'. The first command is 'export OMP_NUM_THREADS=4'. The second command is './count_omp'. The output consists of three lines: 'Count = 1000148', 'Time = 0.04 seconds', and 'Threads used = 4'.

```
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ export OMP_NUM_THREADS=4
huzaifa@huzaifa-VirtualBox:~/openmp_lab$ ./count_omp
Count = 1000148
Time = 0.04 seconds
Threads used = 4
```

Figure 5.1: Output of Exercise 3

5.4 Observation

Critical section ensures correctness but adds overhead, making it slower than reduction.

Chapter 6

Analysis

- OpenMP simplifies parallel programming.
- Reduction is faster than critical section.
- Performance improves with threads but not perfectly linear.

Chapter 7

Conclusion

OpenMP provides an efficient and simple way to implement parallel programs. It improves performance while reducing coding complexity compared to traditional threading methods.